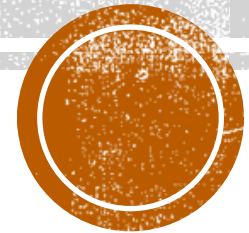




Module: Développer en Front-end

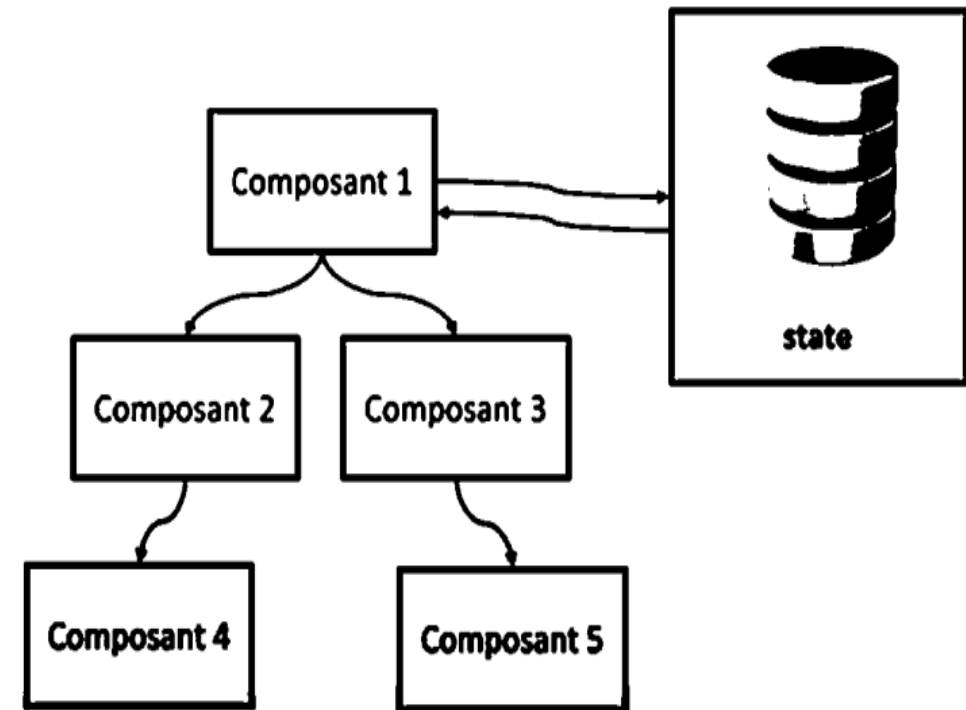
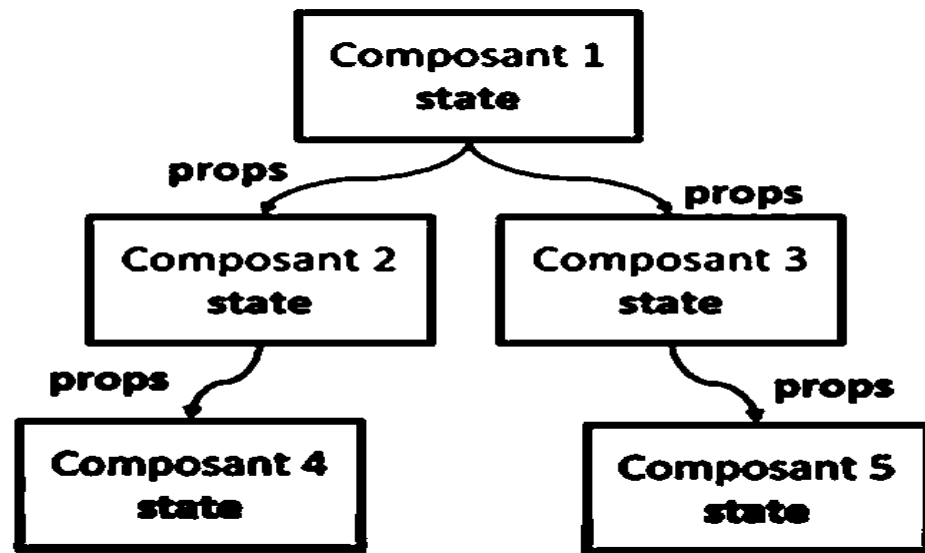
Redux



Filière: Développement digital – option web full stack

Réalisé par: ELMAUDNI

- Redux est une bibliothèque indépendante qui nous aide à gérer notre état en donnant à nos composants l'état dont il a besoin via un store central
- Sans l'utilisation de Redux : state est envoyée via des props à composants parents.



- **Store** : Redux permet de stocker dans un seul objet appelé "the store" tous les états (state) de l'application.

- **Reducer** : Pour mettre à jour le **store**, il n'est pas possible de le faire directement:

On va utiliser la fonction appelée "**reducer**" qui reçoit en arguments le "state" et une "action" et qui retourne le nouveau "state" en fonction de l'"action".

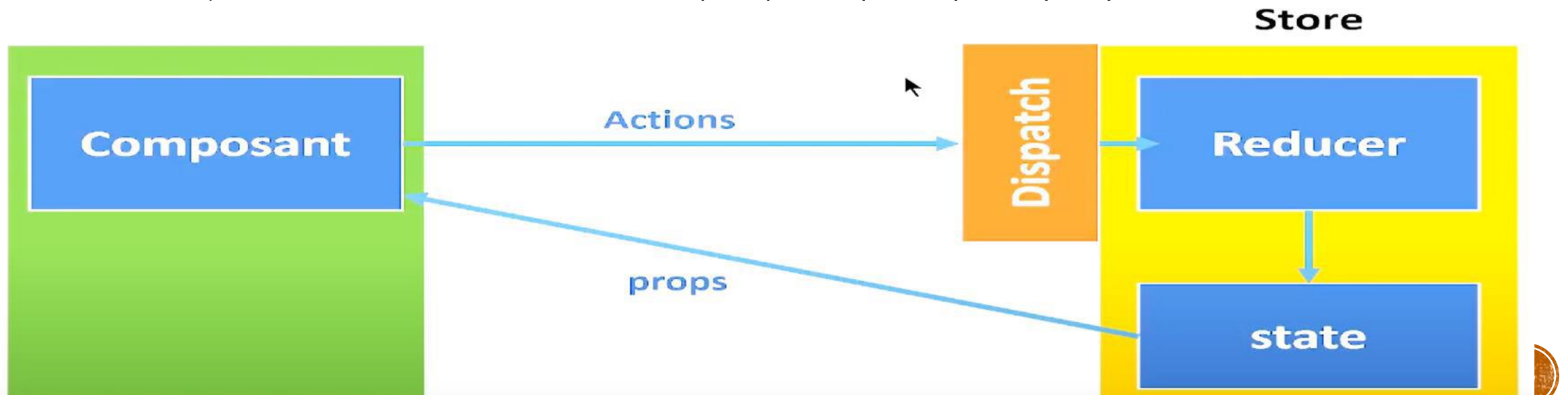
un réducteur est juste une fonction qui reçoit l'état précédent et une action en tant que paramètres et renvoie l'état mis à jour.

- **Action** : Action est un **objet littéral** (plain object) qui représente ce qu'il vient de se passer. On peut l'assimiler à un événement.

Fonctionnement

Voici trois principes fondamentaux de Redux :

- Le state complet de l'application est centralisé en un seul endroit. STORE
- Le 'state' de l'ensemble de l'application est stocké dans une arborescence d'objets au sein d'un seul 'store'
- le 'state' est en lecture seule et nécessite des modifications en émettant une action descriptive.
- les 'state' sont transformé par les actions, via des fonctions 'reducer' pures.
- Pas besoin d'envoyer des 'props' aux composants enfants ou des fonctions 'callback' aux composants parents.
- Avec Redux, le 'state' est centralisé en un seul endroit(store) et chaque component peut y accéder directement.



Etapas:

1-Creation store

- **Création d'un store :**

```
import {legacy_createStore as createStore} from "redux"  
import reducer from "./store/reducer";  
const store=createStore(reducer)
```

Rendre le store disponible pour React

- `import { Provider } from "react-redux";`

```
root.render(<Provider store={store}>  
<App/>  
</Provider>  
);
```

2-Reducer :

- Les fonctions 'reducer' **ne change pas le state actuel**. Il renvoie toujours **un nouvel objet state mis à jour**.

```
const initialState = {  
  articles: [  
    { id: 1, title: "post 1", body: "Quisque cursus, metus vitae pharetra" },  
    { id: 2, title: "post 2", body: "Quisque cursus, metus vitae pharetra" } ] }  
  
const reducer=(state=initialState, action)=>{  
  switch (action.type)  
  {  
    case "ADD":  
      .....  
      return {... state, articles:....., }  
    Default:  
      return state  
  }  
}
```

3- Connecter l'application React à un store:

useSelector et **useDispatch** sont des hooks qui vous permettent d'accéder au store Redux directement dans vos composants React:

Récupérer un élément de store

```
import { useDispatch , useSelector} from "react-redux";  
const articles=useSelector(s=>s.articles)
```

Dans un composant, on peut appeler la fonction 'reducer' en utilisant le hook **useDispatch**

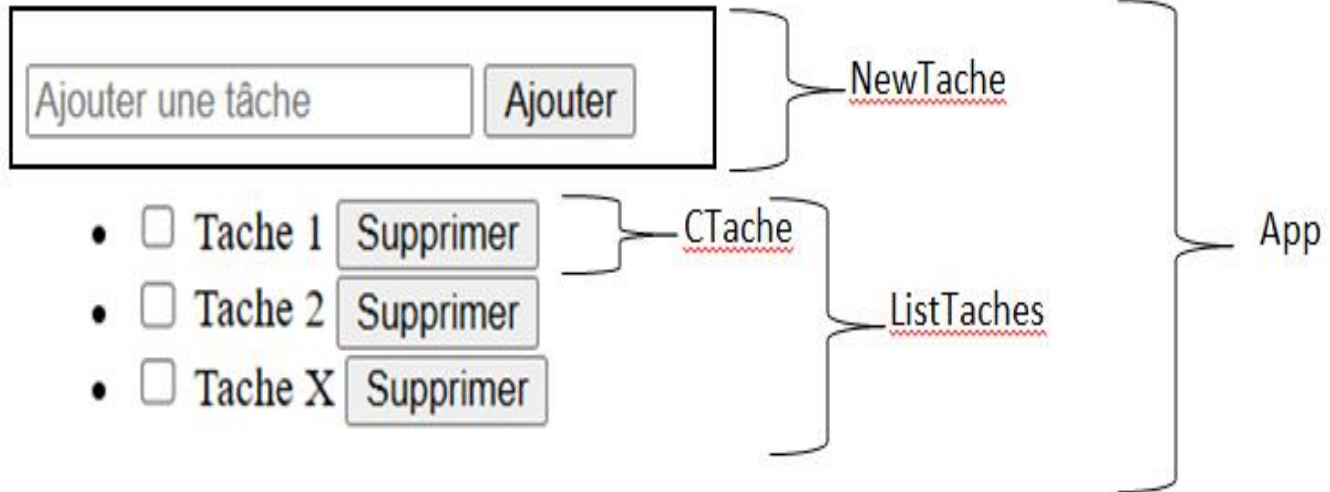
```
const dispatch=useDispatch()
```

Envoyer l'action (type && les données)

```
{{e}=>{dispatch(({type:"ADD", articleData:e}}))}}
```

Exercice d'application :

Gestion des Tâches



Actions créateurs

- les "actions" sont des objets JavaScript qui représentent des événements qui peuvent être déclenchés depuis votre application. Ces actions sont généralement dispatchées à l'aide de la fonction dispatch du store Redux pour mettre à jour l'état global de votre application.

```
// Action Type
const ADD_TACHE = 'ADD_TACHE';

// Action Creator
const addTache= (tache) => {
  return {
    type: ADD_TACHE,
    payload: tache
  };
};
```

Dans cet exemple :

- ADD_TACHE est le type d'action, une chaîne qui décrit l'action.
- addTache est un créateur d'action, une fonction qui crée et retourne un objet représentant l'action à effectuer. Il prend un argument (ici, tache) qui sera envoyé en tant que charge utile (payload) avec l'action.

Exercice Gestion Taches: Actions createurs

- Actions.js

```
// Types d'actions  
const ADD_TACHE = 'ADD_TACHE';  
const SUPPRIMER_TACHE = 'SUPPRIMER_TACHE';  
const MARQUER_TACHE_TERMINEE = 'MARQUER_TACHE_TERMINEE';  
// Action Creators  
const addTache = (tache) => {  
  return { type: ADD_TACHE, payload: tache }; };  
const supprimerTache = (id) => {  
  return { type: SUPPRIMER_TACHE, payload: id }; };  
const marquerTacheTerminee = (id) => {  
  return { type: MARQUER_TACHE_TERMINEE, payload: id }; };
```

Gestion Tache: reducer

```
// Reducer
const initialState = { taches: [ ] };

const reducer = (state = initialState, action) => {
  switch (action.type) {
    case ADD_TACHE:
      return {
        ...state,
        taches: [...state.taches, action.payload]
      };
  }
};
```

```
case SUPPRIMER_TACHE:
  return {
    ...state,
    taches: state.taches.filter((t) => t.id !== action.payload)
  };
case MARQUER_TACHE_TERMINEE:
  return {
    ...state,
    taches: state.taches.map((tache, index) =>
      tache.id === action.payload ? { ...tache, terminee: true }
    : tache
  )
  };
default:
  return state;
}
};
```

Composants: useSelector et useDispatch

```
const ajouter=()=>{  
  let newtache={  
    id:Date.Now(),  
    description:desc,  
    terminer:false  
  }  
  
  dispatch(addTache (newtache));  
}
```

```
const supprimer=(id)=>{  
  
  dispatch(supprimerTache (id));  
  
}
```

```
const marquerTerminer=(id)=>{  
  dispatch(marquerTacheTerminee (id));  
}
```